

Instituto Superior Tecnológico de Movilidad e Innovación

Examen final – Generador inteligente de horarios
académicos (con Nodjs, React, Postgres herramientas
principales)

Nombre:

Edwin Geovany Cusin Antamba

Asignatura:

Matemáticas discretas aplicadas al
desarrollo de software

Docente:

Ing. Richard Heredia

Fecha de entrega:

02 de agosto de 2026

Quito, Ecuador

1. Aplicación de la teoría de conjuntos

El sistema utiliza estructuras Set de JavaScript/TypeScript para representar y comparar materias en distintos momentos del flujo de generación de horarios:

- Conjunto universal (U): todas las materias registradas en la base de datos, obtenidas mediante GET /courses, representan el universo de elementos disponibles para construir horarios.
- Conjunto de un horario (H): cada combinación generada se transforma en un Set de nombres de materias mediante la función `getCourseNameSet(schedule)`, lo que permite operar sobre ella con pertenencia y subconjuntos en lugar de comparar arreglos manualmente.
- Pertenencia ($\in$): se usa `scheduleSet.has(nombre)` para verificar si una materia específica forma parte de un horario generado.
- Subconjunto ($\subseteq$): la función `includesRequiredCourses(scheduleSet, requiredCoursesSet)` valida que todas las materias obligatorias configuradas por el usuario (conjunto O) estén contenidas dentro del horario generado (conjunto H), expresado como $O \subseteq H$.

```
O = {Programación}   H = {Programación, Matemáticas, Inglés}   →   O ⊆ H = true
```

2. Aplicación del álgebra proposicional

Cada regla de negocio del sistema se modela como una proposición booleana (verdadera o falsa), y todas se combinan mediante el operador de conjunción (AND) para determinar si un horario es válido:

- Conjunción (AND / $\wedge$): un horario solo es válido si TODAS las proposiciones individuales son verdaderas al mismo tiempo (cantidad correcta de materias, sin cruces, cumple modalidad, cumple créditos, etc.).
- Negación (NOT / $\neg$): se utiliza para expresar reglas como 'el horario NO tiene cruces' ($\neg P$), a partir del resultado de `hasScheduleConflicts()`.
- Disyunción (OR / $\vee$): aplicada en la validación de modalidad, cuando el horario debe incluir al menos una materia presencial O una virtual, según la configuración.
- Implicación ($\rightarrow$): la validación de prerrequisitos se modela como 'si se selecciona la materia avanzada, entonces debe cumplirse su prerrequisito', implementada mediante la función `meetsPrerequisites()`, que revisa que cada prerrequisito esté presente en el horario o en la lista de materias ya aprobadas por el estudiante.

Cuando un horario no cumple alguna proposición, el sistema no se limita a devolver false: la función `evaluateSchedule()` registra en un arreglo de reasons cuál o cuáles proposiciones fallaron, permitiendo explicar el descarte de forma clara al usuario.

3. Aplicación de la combinatoria

La combinatoria se utiliza en dos niveles complementarios:

- Cálculo del número de combinaciones — la función `calculateCombinationCount(n, r)` implementa la fórmula $C(n,r) = n! / (r! \times (n-r)!)$, apoyada en una función `factorial(n)` escrita de forma iterativa, para conocer de antemano cuántos horarios distintos son matemáticamente posibles antes de generarlos.
- Generación real de combinaciones — la función `generarCombinaciones(elementos, cantidadSeleccionar)` implementa un algoritmo de backtracking recursivo que construye cada combinación posible, agregando y removiendo elementos del arreglo actual hasta cubrir todas las selecciones válidas, sin repetir grupos ni considerar el orden (es decir, $\{A, B\}$ y $\{B, A\}$ se tratan como el mismo horario).

Se utilizan combinaciones y no permutaciones porque, en el contexto de un horario académico, el orden en que se listan las materias seleccionadas no genera un horario distinto; lo relevante es únicamente qué materias lo componen.

4. Regla lógica principal

La regla completa que determina si un horario es válido combina las siete condiciones del sistema mediante conjunción lógica:

$T \wedge O \wedge C \wedge M \wedge D \wedge R \wedge P$

T	Tiene la cantidad correcta de materias
O	Incluye las materias obligatorias ($O \subseteq H$)
C	No tiene cruces de horario ($\neg \text{cruce}$)
M	Cumple la modalidad requerida
D	No supera el máximo de materias difíciles
R	No supera el máximo de créditos
P	Cumple los prerrequisitos (implicación $\rightarrow$)

5, 6 y 7. Resultados de una ejecución real del sistema

Configuración utilizada: 10 materias disponibles, horarios de 3 materias, con la materia "Programación" como obligatoria y las 7 reglas del sistema activas.

The screenshot shows a web interface titled "CÁLCULO COMBINATORIO" with a subtitle "Combinatoria: cuántos grupos distintos de materias son posibles, sin importar el orden." It displays the following data:

Input	Value
Materias disponibles	10
Materias por horario	3
Combinaciones posibles	$C(10,3) = 120$
Horarios válidos	13
Horarios descartados	107

De las 120 combinaciones matemáticamente posibles según $C(10,3)$, únicamente 16 cumplieron simultáneamente las 7 proposiciones de la regla lógica principal ($T \wedge O \wedge C \wedge M \wedge D \wedge R \wedge P$). Las 104 combinaciones restantes fueron descartadas por incumplir una o varias condiciones, y cada una conserva registrado el motivo exacto del descarte (por ejemplo: cruce de horario, superar el máximo de créditos, no incluir la materia obligatoria, o no cumplir un prerrequisito).

Conclusión

El desarrollo de GIHA demuestra de forma práctica que la teoría de conjuntos, el álgebra proposicional y la combinatoria no son conceptos aislados de las matemáticas discretas, sino herramientas directamente aplicables a la construcción de software real: desde el modelado de datos en PostgreSQL, pasando por la lógica de validación en el backend con Node.js y Prisma, hasta la presentación de los resultados y su fundamento matemático en la interfaz de React.